KnowFeat: Knowledge-Guided Feature Engineering with Auditable Provenance via LLM Agents

Chengsong You[1], Wangyue Li[1], Weiqiao Que[1], Qizhou Chen[1], Wei Deng[2], Feng Zhu[3], Xiaofeng He[1,*]
[1]East China Normal University, Shanghai, China
{51275901122, 52285901017}@stu.ecnu.edu.cn, weiqiaoque@gmail.com, 582476065@qq.com, hexf@cs.ecnu.edu.cn
[2]Southwestern University of Finance and Economics, Chengdu, China
dengwei@swufe.edu.cn
[3]Digital Currency Department, Postal Savings Bank of China, Beijing, China
Jason_bank@126.com
*Corresponding author: hexf@cs.ecnu.edu.cn

***Abstract*—Automated feature engineering with large language models (LLMs) can generate semantically meaningful features for tabular data, yet existing methods lack structured domain knowledge, rigorous verification, and auditable provenance. We propose KnowFeat, a knowledge-guided feature engineering framework that addresses all three gaps. KnowFeat organizes domain knowledge into a five-type hierarchy—schema metadata, regulatory indicators, detection rules, expert opinions, and case evidence—and injects it into an LLM agent. A three-stage verification pipeline (L1–L3) filters candidates through code execution, statistical quality checks, and model evaluation. Every accepted feature carries a provenance card tracing its design to specific knowledge assets. On seven public benchmarks, KnowFeat achieves the best AUC on 4 of 7 datasets and the best average rank (1.4) among seven methods. On a real-world anti-money laundering (AML) dataset with 200K transactions, 18 knowledge-guided features improve Recall by +2.1pp and F1 by +1.5pp over a 93-feature baseline, outperforming methods that generate up to 1,031 unverified features. Ablation studies confirm that hierarchical domain knowledge is essential for consistent feature quality.**

***Index Terms*—Feature Engineering, Large Language Models, Domain Knowledge, Anti-Money Laundering, Tabular Data**

I. Introduction

Feature engineering is widely recognized as one of the most impactful steps in building machine learning models for tabular data [1]. Carefully designed features can substantially improve model accuracy, reduce overfitting, and provide interpretable signals for downstream tasks. However, manual feature engineering is labor-intensive, requires deep domain expertise, and does not scale.

Traditional automated feature engineering (AutoFE) methods such as OpenFE [2] and AutoFeat [3] explore a predefined transformation space without understanding what features *mean*. The emergence of large language models (LLMs) has enabled semantic feature generation: CAAFE [4], FeatLLM [5], and LLM-FE [6] use LLMs to generate feature code from dataset descriptions. However, these methods share three fundamental limitations:

(1) Lack of structured domain knowledge. Existing methods rely on the LLM's parametric knowledge, which may be outdated or incomplete for specialized domains. None organizes external knowledge into a hierarchical format for systematic injection, yet such structured knowledge is abundant in regulated domains like anti-money laundering (AML) [7], [8].

(2) Insufficient verification. CAAFE uses a logistic regression proxy, LLM-FE uses downstream accuracy without statistical filtering, and FeatLLM performs no model-based validation. None combines code execution, statistical quality filtering, and model evaluation into a unified pipeline.

(3) Limited auditability. In regulated industries, features must be auditable—decision-makers need to understand *why* a feature was created and *what evidence* supports it. No existing method provides structured provenance tracing each feature to specific knowledge assets.

We propose **KnowFeat** (**Know**ledge-guided **Feat**ure engineering), a framework that addresses all three limitations:

- **Hierarchical Knowledge Injection.** We organize domain knowledge into five types of increasing specificity—schema metadata, risk indicators, detection rules, expert opinions, and case evidence—and inject them as structured context into the LLM agent.
- **Three-Stage Verification Pipeline.** Each candidate passes through L1 (code execution sandbox), L2 (statistical quality with noise-calibrated thresholds), and L3 (model effectiveness via XGBoost across 5 seeds).
- **Auditable Provenance.** Every accepted feature carries a provenance card recording the knowledge assets, reasoning chain, computation steps, and verification results.

We evaluate KnowFeat on seven public benchmarks from OpenML and SimECNY, a real-world AML dataset with 200,000 digital currency transactions and 93 baseline features. Key findings:

- On public datasets, KnowFeat achieves the best AUC on 4 of 7 datasets and the best average rank (1.4) among seven methods, with significantly fewer features than LLM-based alternatives.
- On SimECNY, 18 knowledge-guided features improve Recall by +2.1pp and F1 by +1.5pp over the 93-feature baseline, outperforming all baselines.

TABLE I
COMPARISON OF AUTOMATED FEATURE ENGINEERING METHODS. ✓: FULLY SUPPORTED; ○: PARTIALLY SUPPORTED; BLANK: NOT SUPPORTED; –: NOT APPLICABLE.

Property	OpenFE	AutoF.	CAAFE	FeatL.	LLM-FE	RiskT.	Ours
Semantic underst.			✓	✓	✓	✓	✓
Hier. knowledge					○[1]	○[2]	✓
Exec. verific.	–	–					✓
Statistical filt.	✓	✓					✓
Model validation			○[3]		✓		✓
Provenance track.						○[4]	✓
Adaptive search					✓		✓

[1]Dataset desc. only. [2]Regulatory guidelines only. [3]LogReg proxy. [4]NL rationales only.

- Ablation confirms that domain knowledge is essential: the schema-only variant generates zero accepted features on one dataset.

II. Related Work

A. Automated Feature Engineering

Traditional AutoFE methods search a *syntactic* transformation space. Deep Feature Synthesis [9] applies aggregation primitives along relational paths; ExploreKit [10] enumerates operators via meta-learning; AutoFeat [3] uses polynomial expansion with L1 selection; and OpenFE [2] frames generation as operator-operand search with LightGBM-based ranking. AutoML systems such as Auto-sklearn [11] and FLAML [12] include preprocessing but do not generate semantically new features. These methods apply predefined operators without *semantic* understanding [13].

The emergence of LLMs [14], [15] enabled semantic feature generation. CAAFE [4] iteratively prompts an LLM to generate Python code using logistic regression feedback. FeatLLM [5] extracts rules and converts them to binary indicators. LLM-FE [6] uses evolutionary optimization with model performance feedback. RiskTagger [16] uses regulatory knowledge for Web3 behavior annotation. Tool-augmented agents [17] have been applied to code generation [18], software engineering [19], and data science [20]–[22], but none addresses knowledge-grounded feature generation.

These methods vary in domain knowledge and verification depth. Table I summarizes the key differences. KnowFeat is the first to combine hierarchical structured knowledge, multi-stage verification (L1–L3), and full provenance tracking.

In financial AML, domain experts manually encode risk indicators and network measures [23], regulatory bodies publish red-flag typologies [7], and institutions maintain detection rules from compliance experience [8]. Knowledge-infused learning [24] and transfer learning [25] target unstructured data rather than tabular features. This rich, structured AML knowledge has not been systematically leveraged by automated feature engineering.

III. Methodology

This section presents the KnowFeat framework in detail. We first provide an overview of the pipeline (Section III-A), then describe each component: hierarchical knowledge injection (Section III-B), the LLM agent with specialized tools (Section III-C), the three-stage verification pipeline (Section III-D), the intelligent search strategy (Section III-E), and auditable provenance output (Section III-F).

A. Framework Overview

KnowFeat is a multi-round, knowledge-guided feature engineering framework built on an LLM agent. Given a tabular dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ with baseline features $\mathbf{x}_i \in \mathbb{R}^d$ and target label y_i, along with a domain knowledge base $\mathcal{K}$, KnowFeat generates a set of new features $\mathcal{F}^* = \{f_1, f_2, \ldots, f_k\}$ such that the augmented dataset $\mathcal{D}' = \{([\mathbf{x}_i;\ f_1(\mathbf{x}_i); \cdots ;\ f_k(\mathbf{x}_i)],\ y_i)\}$ achieves improved predictive performance on a target metric (e.g., AUC [26] or Recall).

As illustrated in Figure 1, the pipeline proceeds in four phases. First, the system loads the dataset, computes baseline performance, and calibrates statistical thresholds via noise injection (Section III-D). Second, domain knowledge $\mathcal{K}$ is organized into a five-type hierarchy and injected into the agent's context (Section III-B). Third, the agent performs multi-round generation: in each round, per-dimension budgets are allocated based on SHAP feedback, the agent proposes features grounded in knowledge assets, and each candidate passes through the L1→L2→L3 verification pipeline. Accepted features augment the baseline; rejected ones enter the failure memory. Forward selection further recovers individually weak but collectively effective feature combinations. The process terminates when no new features are accepted for $C = 2$ consecutive rounds or $R_{\max}$ rounds are reached. Finally, the system outputs the accepted feature set $\mathcal{F}^*$ with executable code and auditable provenance cards (Section III-F).

B. Hierarchical Knowledge Injection

A key innovation of KnowFeat is the systematic integration of external domain knowledge into the feature generation process. We organize knowledge into five hierarchical types of increasing specificity, from general metadata to concrete case evidence:

(i) Schema Metadata. The agent receives only the dataset schema—column names, data types, and basic statistics (mean, cardinality, missing rate). No domain knowledge is provided. This serves as the ablation baseline, testing what the LLM can generate from metadata alone.

(ii) Risk Indicators. Structured risk indicators derived from regulatory guidelines and industry standards. Each indicator specifies a risk dimension, risk class, relevant data field, and directionality. For the AML domain, we compile 38 indicators (IND_01 through IND_38) covering transaction amount thresholds, frequency patterns, temporal anomalies, and network centrality measures. For example, IND_05 flags

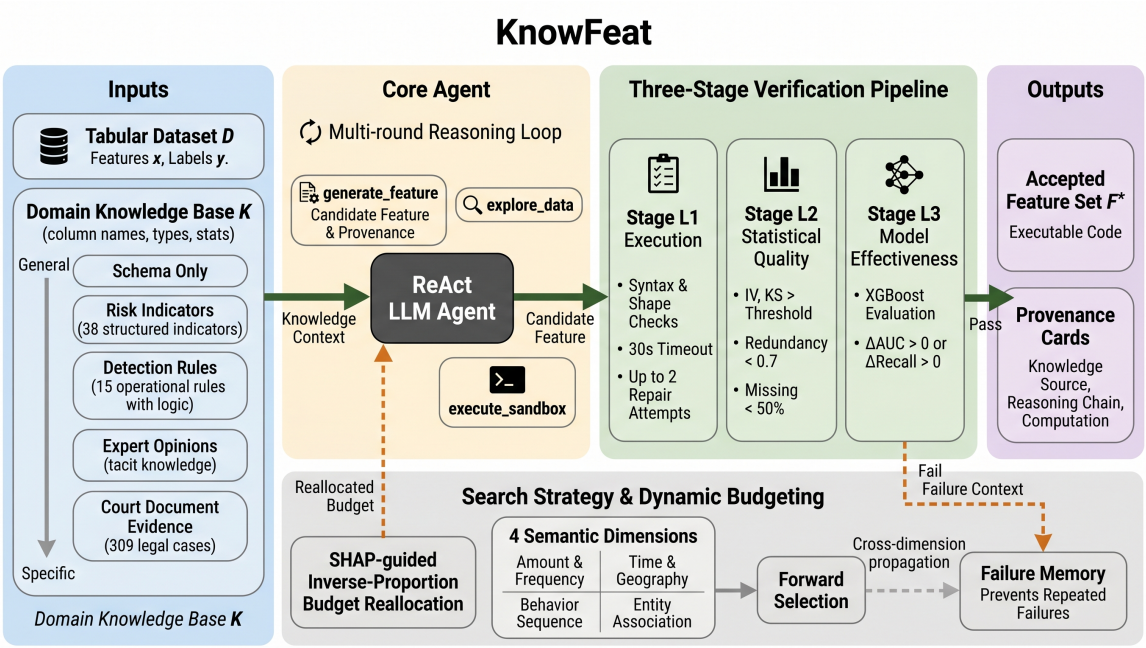


Fig. 1. Overview of the KnowFeat framework. Color coding: blue = inputs (data & knowledge), yellow = core LLM agent, green = verification pipeline (L1–L3), purple = outputs (features & provenance cards), gray = SHAP-guided search strategy. The agent generates features grounded in hierarchical domain knowledge; each candidate is verified through a three-stage pipeline; SHAP feedback dynamically reallocates the exploration budget across semantic dimensions.

accounts whose single-day transaction count exceeds a regulatory threshold, while IND_15 identifies abnormal transaction modes such as fan-in and many-to-many patterns.

(iii) Detection Rules. Operational detection rules that encode specific logic patterns. Each rule includes a rule ID, natural language description, logical expression, confidence level, and a Python implementation hint. We compile 15 rules (RULE_001 through RULE_015) from compliance manuals and internal documentation. For example, RULE_006 encodes the "scatter-in, gather-out" layering pattern.

(iv) Expert Opinions. Insights from domain expert interviews that capture tacit knowledge—heuristics, edge cases, and domain intuitions—difficult to formalize as rules but valuable for feature design. We collect 12 expert opinions (EXP_01 through EXP_12). For example, EXP_05 notes that "border region transactions combined with nighttime activity are a hallmark of cross-border laundering."

(v) Court Document Evidence. Risk factors extracted from 309 real court judgments via a key-clue extraction pipeline. This is the most concrete type, grounding feature generation in case evidence (e.g., pyramid-shaped fund network patterns independently validated by legal proceedings).

The five types can be freely combined via a knowledge filter parameter, enabling ablation study of each type's contribution (Section IV-D). Each knowledge item is assigned a unique identifier for provenance tracking. Table II illustrates one representative example per type from the AML domain.

TABLE II
REPRESENTATIVE KNOWLEDGE EXAMPLES FROM THE AML DOMAIN. EACH TYPE ADDS SPECIFICITY: FROM RAW SCHEMA TO CASE-GROUNDED EVIDENCE.

Type	Count	Example
Schema	—	`amount` (numeric), `wallet_type` (4 categories), `hour` (0–23)
Indicator	38	**IND_05**: Daily tx count exceeds threshold → money mule risk
Rule	15	**RULE_006**: Scatter-in, gather-out; daily in-count ≥20, total ≥1M
Expert	12	**EXP_05**: Border region + nighttime = cross-border laundering signal
Case evid.	309	Multi-bank wallet network for layered fund channeling (court judgment)

C. LLM Agent with Specialized Tools

KnowFeat employs an LLM agent (DeepSeek-V3 in our implementation) that interleaves reasoning with tool calls following the ReAct paradigm [17]. The agent is equipped with three specialized tools:

Tool 1: Data Exploration. Executes Python code on the dataset in a sandboxed environment to inspect distributions, compute correlations, and identify patterns before proposing features. The tool returns printed output or return values to the agent.

Tool 2: Feature Registration. Registers a candidate feature with mandatory provenance fields:

- `feature_name`: A descriptive identifier.
- `code`: Executable Python code that computes the feature

from raw data.

- `explanation`: A natural language description of what the feature captures.
- `reasoning`: A structured chain linking knowledge assets to the feature design, formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]..."
- `knowledge_ids`: A list of specific knowledge asset identifiers (e.g., RULE_001, IND_05, EXP_03) that inspired the feature.
- `computation_steps`: Step-by-step description from raw fields to the final feature value.
- `data_fields_used`: The raw columns consumed by the feature.

Tool 3: Verification Trigger. Triggers the L1→L2→L3 verification pipeline on a registered feature and returns the result (accepted/rejected with detailed reasons). The agent must call Feature Registration followed immediately by Verification Trigger for each candidate—batch submission is prohibited to ensure tight feedback loops.

The agent's prompt instructs it to: (1) reason about which knowledge assets are relevant to the current dimension and budget, (2) explore the data to validate assumptions, (3) propose a feature grounded in specific knowledge items, and (4) submit it for verification and react to the feedback.

D. Three-Stage Verification Pipeline

Each candidate feature passes through three progressively stricter verification stages:

1) L1: Code Execution Sandbox: The generated Python code is executed in an isolated sandbox with a 30-second timeout. L1 checks for:

- Syntax and runtime errors.
- Output shape consistency (one value per sample).
- Presence of required identifier columns.
- Data type validity (numeric output).

If L1 fails, the agent receives the error message and is allowed up to 2 automatic repair attempts before the feature is permanently rejected.

2) L2: Statistical Quality Filtering: The feature's statistical properties are evaluated against automatically calibrated thresholds. To establish these thresholds for discriminative power, we generate $N_{\text{noise}} = 100$ random features (drawn from standard distributions), compute their quality metrics, and set the acceptance threshold at the P_{noise}-th percentile (default $P_{\text{noise}} = 95$). This ensures that accepted features are statistically better than random noise. L2 checks four criteria:

- **Information Value (IV)** [23]: Must exceed the noise-calibrated threshold, ensuring the feature has meaningful discriminative power between positive and negative classes.
- **Kolmogorov–Smirnov (KS) Statistic** [13]: Must exceed the noise-calibrated threshold, measuring the maximum distributional separation between classes.
- **Redundancy**: Maximum absolute Pearson correlation with all existing accepted features must be below 0.7, preventing near-duplicate features from inflating the feature set.
- **Missing Rate**: The proportion of NaN values must be at most 50%, ensuring sufficient coverage for model training.

The noise-calibrated thresholds apply to IV and KS; the redundancy and missing rate thresholds are fixed.

3) L3: Model Effectiveness: The feature is evaluated for its actual contribution to predictive performance. We train an XGBoost [27] model on the baseline features augmented with the candidate feature, using stratified 80/20 train-test splits across 5 random seeds $\{42, 123, 456, 789, 2024\}$. The acceptance criterion depends on the task:

- **Classification (public datasets)**: $\Delta\text{AUC} > 0$, i.e., the augmented model must achieve strictly higher AUC than the baseline.
- **AML detection**: $\Delta\text{Recall} > 0$ *and* $\text{Precision} \geq 90\%$, ensuring that the feature improves detection without sacrificing precision.

The three-stage design is intentionally progressive: L1 is cheap (milliseconds), L2 is moderate (seconds), and L3 is expensive (minutes per feature). By filtering early, we avoid wasting computational resources on clearly defective candidates.

E. Intelligent Search Strategy

To systematically explore the feature space, KnowFeat employs a multi-component search strategy.

1) Dimension-Based Budgeting: We decompose the feature space into D semantic dimensions relevant to the target domain (default $D = 4$ for the AML domain):

1) **Amount & Frequency**: Features related to monetary values and transaction counts.
2) **Time & Geography**: Features capturing temporal patterns and spatial distributions.
3) **Behavior Sequence**: Features encoding sequential patterns in user actions.
4) **Entity Association**: Features derived from relationships between entities (accounts, counterparties).

Each dimension receives an initial budget of B_0 candidates per round (default $B_0 = 3$). After each round, SHAP [28] (SHapley Additive exPlanations) analysis is performed on the current model to estimate each dimension's contribution to predictions.

2) SHAP-Guided Budget Reallocation: Let s_j denote the mean absolute SHAP value for dimension j's accepted features, $B_{\min}$ the minimum per-dimension budget (default 1), and B_{total} the total budget for the round. We reallocate the budget for round $r{+}1$ using an *inverse-proportion* strategy:

$$B_j^{(r+1)} = B_{\min} + (B_{\text{total}} - D \cdot B_{\min}) \cdot \frac{1/s_j}{\sum_{j'=1}^{D} 1/s_{j'}} \quad (1)$$

The intuition is that dimensions already well-represented in the model (high s_j) need less exploration, while underrepresented dimensions receive more budget, promoting balanced feature

coverage and preventing the agent from over-exploiting a single dimension.

3) Failure Memory: A failure memory $\mathcal{M}_{\text{fail}}$ records all rejected features along with their rejection reasons. Before each generation step, the K most recent failures for the current dimension (default $K = 10$) are injected into the agent's prompt, preventing the regeneration of previously rejected ideas and guiding the agent toward novel directions.

4) Forward Selection: Individual features may fail L3 yet contribute positively when combined with others. We apply greedy forward selection on L2-passed but L3-failed candidates, sorted by L2 quality scores. Each candidate is added to the accepted set if the augmented set improves the target metric, capturing synergistic feature interactions.

5) Cross-Dimension Propagation: After each round, the top-K_{cross} features (default $K_{\text{cross}} = 5$) by SHAP importance are shared across all dimensions as "inspiration prompts," encouraging the agent to adapt successful patterns from one dimension to others.

F. Auditable Provenance Output

A distinguishing contribution of KnowFeat is that every generated feature is accompanied by a structured *provenance card* that enables full auditability. Each provenance card records:

- **Knowledge assets**: The specific identifiers (e.g., RULE_006, IND_15, EXP_05) that inspired the feature design.
- **Reasoning chain**: A step-by-step explanation of how domain knowledge led to the feature, formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]..."
- **Computation steps**: The transformation pipeline from raw data fields to the final feature value.
- **Verification results**: L1/L2/L3 outcomes including IV, KS, redundancy, and ΔAUC/ΔRecall.

This traceability is essential for regulated domains: compliance officers can audit *why* each feature exists, *what evidence* supports it, and *how* it was verified. Section V illustrates provenance cards through concrete examples.

IV. Experiments

We evaluate KnowFeat on both public tabular benchmarks and a real-world anti-money laundering (AML) dataset. Our experiments aim to answer four research questions:

- **RQ1**: Does KnowFeat improve predictive performance on public datasets?
- **RQ2**: How does KnowFeat perform on a real-world AML task with severe class imbalance?
- **RQ3**: What is the contribution of domain knowledge?
- **RQ4**: How does KnowFeat balance feature quantity vs. quality?

TABLE III
Dataset statistics.

Dataset	Samples	Feat.	Pos%	Domain
credit_g	1,000	20	70.0	Finance
diabetes	768	8	34.9	Healthcare
blood_transfusion	748	4	23.8	Healthcare
heart	918	11	55.3	Healthcare
bank_marketing	45,211	16	11.7	Marketing
breast_w	683	9	35.0	Healthcare
adult	48,842	14	23.9	Census
SimECNY (AML)	200,000	93	2.24	Finance

A. Experimental Setup

1) Datasets: We use eight datasets summarized in Table III: seven public benchmarks from OpenML [26] and one real-world AML dataset.

The seven public datasets are standard tabular classification benchmarks obtained via the OpenML Python API [26]. SimECNY is a real-world digital currency transaction dataset containing 200,000 transactions with 93 baseline features and 4,484 positive (suspicious) samples, representing a realistic AML scenario with severe class imbalance (2.24% positive rate).

2) Baselines: We compare against six baselines:

- **No-FE**: XGBoost [27] on original features.
- **OpenFE** [2]: Transformation-based AutoFE with LightGBM [29]-based selection.
- **AutoFeat** [3]: Polynomial feature generation with L1 selection.
- **CAAFE** [4]: LLM-based iterative feature generation with logistic regression feedback.
- **FeatLLM** [5]: LLM-based rule extraction and binary feature generation.
- **LLM-FE** [6]: Evolutionary LLM feature optimization with structured prompts.

3) Evaluation Protocol: All methods are evaluated using XGBoost [27] (n_estimators=200, max_depth=6, learning_rate=0.1) with stratified 80/20 train-test splits across 5 random seeds. We use AUC as the primary metric for all datasets. For SimECNY, we additionally report Recall, Precision, and F1 with scale_pos_weight=auto ($\approx$43.6) to handle class imbalance. All baselines are reproduced using our unified evaluation protocol to ensure fair comparison.

4) Computational Cost: KnowFeat's cost is dominated by LLM API calls and L3 model evaluation. On public datasets, the average wall-clock time is 15–40 minutes per dataset (DeepSeek-V3 API), with an average API cost of $2.40 per run. L3 verification accounts for $\sim$60% of total time. On SimECNY (200K samples), a full run takes $\sim$2 hours. By comparison, LLM-FE requires 3–8 hours due to its evolutionary search, while OpenFE completes in 5–15 minutes but generates only syntactic transformations.

B. RQ1: Public Dataset Performance

Table IV presents the AUC comparison on seven public datasets.

KnowFeat achieves the best performance on 4 of 7 datasets (credit_g, blood_transfusion, breast_w, adult) and ranks second on 2 datasets (diabetes, heart). OpenFE achieves the best results on 3 datasets through exhaustive operator-operand search but generates 34–50 features per dataset, while KnowFeat achieves competitive or superior performance with 12–46 verified features. Among LLM-based methods, LLM-FE generally outperforms CAAFE and FeatLLM but remains below both OpenFE and KnowFeat. Notably, LLM-FE underperforms No-FE on two datasets, suggesting that evolutionary optimization without structured knowledge and multi-stage verification is insufficient.

Takeaway: KnowFeat achieves the best average rank (1.4) with the most consistent performance across diverse domains, without requiring dataset-specific tuning.

C. RQ2: AML Dataset Performance

Table V presents results on the SimECNY AML dataset with scale_pos_weight=auto. KnowFeat adds 18 knowledge-guided features to the 93-feature baseline, achieving the best results across all four metrics.

KnowFeat's 18 domain-knowledge features yield substantial improvements: Recall improves by +2.1pp over No-FE (0.987 vs. 0.966), F1 by +1.5pp (0.986 vs. 0.971), and AUC reaches 0.9999. LLM-FE achieves the second-best F1 (0.973) with only 1 additional feature, but KnowFeat's margin over LLM-FE remains large (+1.3pp F1, +1.7pp Recall). FeatLLM adds 1,031 features yet achieves the *worst* performance, demonstrating that unverified feature quantity degrades rather than improves models.

Robustness check. We re-evaluate with scale_pos_weight=1 (no class balancing). KnowFeat remains the clear winner (F1 0.981) and is the *only* method that outperforms No-FE, confirming that the 18 features encode genuine discriminative signal independent of class reweighting.

Takeaway: On the real-world AML dataset, KnowFeat's 18 verified features outperform 1,031 unverified features from FeatLLM, demonstrating that knowledge-guided quality dominates brute-force quantity.

D. RQ3: Ablation Study

To quantify the contribution of domain knowledge, we compare the *schema_only* variant (schema metadata only) against the *full* variant (all five knowledge types). Results on public datasets are shown in Table VI.

On diabetes, schema-only generates *zero* features passing L3, while full knowledge produces 24 features (+0.016 AUC). On five of six remaining datasets, the full variant outperforms schema-only. On SimECNY, the gap is largest: schema-only generates 3 features (+0.3pp Recall) vs. full knowledge with 18 features (+2.1pp Recall), confirming that AML-specific knowledge is the primary driver.

Takeaway: Domain knowledge is essential—without it, the LLM generates zero useful features on one dataset and consistently fewer/weaker features on others.

E. RQ4: Feature Quantity vs. Quality

Table VII compares the number of features generated by each method.

FeatLLM generates the most features (98–1,124) yet achieves the worst performance on most datasets. LLM-FE generates few features (9–21) but cannot consistently outperform No-FE. KnowFeat strikes an effective balance: it generates a moderate number of verified features (12–46 on public datasets, +18 on SimECNY) that have all passed the L1→L2→L3 pipeline. On SimECNY, KnowFeat's 18 verified features improve F1 by +1.5pp, while FeatLLM's 1,031 unverified features *decrease* F1 by -0.4pp.

Takeaway: KnowFeat's three-stage verification acts as a quality filter, producing fewer but more effective features than methods that maximize quantity.

V. Case Study: Knowledge-Grounded Features

We present two representative features from the SimECNY AML dataset to illustrate multi-source knowledge fusion and provenance traceability. Figure 2 shows an example provenance card.

Case 1: Fan-In/Fan-Out Ratio. This feature captures network topology asymmetry by fusing four knowledge assets spanning three types: *RULE_006* (scatter-in/gather-out layering), *RULE_013* (pyramid fund networks), *PAT_A03* (money mule pattern), and *IND_15* (abnormal transaction modes). The reasoning chain: because RULE_006/013 indicate directional imbalance in fund flows, compute the fan-in/fan-out mode ratio per wallet; because IND_15 flags abnormal modes, weight by abnormal mode proportion: ratio $\times$ (1 + abnormal_mode_ratio). No syntactic method would produce this feature—it requires understanding that directional imbalance combined with abnormal mode prevalence characterizes money mule behavior.

Case 2: Cross-Dimensional Risk Interaction. This feature multiplies geographic risk with temporal anomaly, guided by three knowledge assets: *RULE_005/014* (cross-regional border flows), *IND_08/09* (off-hour concentration), and *EXP_05* ("border region + nighttime = cross-border laundering"). Neither geographic nor temporal signal alone is sufficient; the expert opinion prescribes their conjunction: is_high_risk_region $\times$ off_hour_ratio. This demonstrates *cross-dimensional interaction*: geographic context combined with temporal patterns, where the multiplicative form ensures only the conjunction triggers a high score.

Both features carry complete provenance chains for regulatory audit, a capability absent from all baselines.

VI. Conclusion

We presented KnowFeat, a knowledge-guided feature engineering framework that combines hierarchical domain knowledge injection, multi-stage verification (L1–L3), and auditable

TABLE IV
AUC COMPARISON ON PUBLIC DATASETS (MEAN±STD OVER 5 SEEDS). **BOLD**: BEST; UNDERLINE: SECOND BEST. T/O = TIMEOUT (>24H).

Dataset	No-FE	OpenFE	AutoFeat	CAAFE	FeatLLM	LLM-FE	KnowFeat
credit_g	.767±.021	.783±.031	.754±.017	.760±.023	.760±.022	.762±.026	**.785±.014**
diabetes	.803±.033	**.825±.034**	.809±.027	.803±.030	.810±.032	.815±.033	.819±.032
blood_transfusion	.682±.043	.665±.040	.682±.043	.678±.043	.681±.044	.665±.045	**.701±.036**
heart	.932±.007	**.942±.011**	.934±.003	.930±.006	.930±.004	.933±.005	.938±.006
bank_marketing	.933±.002	**.937±.002**	T/O	.933±.002	.934±.002	.932±.002	.934±.002
breast_w	.993±.005	.994±.005	.994±.004	.992±.006	.992±.005	.994±.001	**.996±.003**
adult	.929±.004	.930±.003	.925±.004	.929±.003	.929±.003	.928±.003	**.930±.003**
Avg. Rank	4.5	2.1	5.1	5.5	4.6	4.7	**1.4**

TABLE V
SIMECNY AML RESULTS (SPW=AUTO, 5 SEEDS). n_f: TOTAL FEATURE COUNT INCLUDING ORIGINAL 93 BASELINE FEATURES.

Method	n_f	Recall	Prec.	F1	AUC
No-FE	93	.966±.010	.976±.004	.971±.005	.9996
OpenFE	123	.965±.008	.975±.004	.970±.003	.9996
CAAFE	94	.968±.008	.973±.006	.970±.005	.9996
FeatLLM	1124	.963±.010	.973±.005	.968±.006	.9995
LLM-FE	94	.970±.007	.976±.004	.973±.005	.9996
Ours	**111**	**.987**±.006	**.986**±.001	**.986**±.003	**.9999**

TABLE VI
ABLATION: SCHEMA_ONLY VS. FULL KNOWFEAT (AUC).

Dataset	schema_only	full	Δ
credit_g	.775±.012	**.785**±.014	+.010
blood_trans.	.689±.048	**.701**±.036	+.012
heart	.930±.005	**.938**±.006	+.008
bank_mkt.	.931±.002	**.934**±.002	+.003
adult	.929±.003	**.930**±.003	+.001
breast_w	**.996**±.002	.996±.003	−.001
diabetes	— (0 feat.)	.819±.032	—

TABLE VII
NUMBER OF FEATURES (INCL. ORIGINAL). T/O = TIMEOUT.

Dataset	No-FE	Open. FE	Auto. Feat	CAA. FE	Feat. LLM	LLM- FE	Ours
credit_g	20	50	23	24	280	21	46
diabetes	8	38	13	9	152	15	24
blood_trans.	4	34	4	7	120	9	12
heart	11	41	21	14	98	19	31
bank_mkt.	16	46	T/O	21	132	20	36
breast_w	9	39	20	10	107	15	16
adult	14	44	165	19	170	21	34
SimECNY	93	123	—	94	1124	94	111

provenance within an LLM agent. On seven public datasets, KnowFeat achieves the best AUC on 4 of 7 and the best average rank (1.4) among seven methods, while requiring significantly fewer features than alternative LLM-based methods. On a real-world AML dataset with 200K transactions, 18 knowledge-guided features improve Recall by +2.1pp and F1 by +1.5pp over a strong 93-feature baseline—gains that are robust across different class balancing settings. The ablation study confirms that domain knowledge is essential, and every generated feature is traceable to specific knowledge assets.

Limitations. The current knowledge hierarchy is manually organized by domain experts, incurring setup cost for new domains. KnowFeat uses a single LLM for all reasoning; a multi-agent architecture could improve efficiency. The LLM API cost averages $2.40 per dataset run (see Section IV-B).

Future Work. Automating knowledge extraction from heterogeneous sources (regulations, case law, expert interviews) and extending KnowFeat to additional high-stakes domains such as cybersecurity and insurance fraud detection would further validate the generality of the knowledge-guided paradigm.

REFERENCES

[1] P. Domingos, "A few useful things to know about machine learning," *Communications of the ACM*, vol. 55, no. 10, pp. 78–87, 2012.

[2] T. Zhang, Z. A. Zhang, Z. Fan, H. Luo, F. Liu, Q. Liu, W. Cao, and L. Jian, "OpenFE: Automated feature generation with expert-level performance," in *Proc. of the 40th Int'l Conf. on Machine Learning (ICML)*, 2023, pp. 41 880–41 901.

[3] F. Horn, R. Pack, and M. Rieger, "The autofeat Python library for automated feature engineering and selection," in *Machine Learning and Knowledge Discovery in Databases (ECML-PKDD)*. Springer, 2019, pp. 111–120.

[4] N. Hollmann, S. Müller, and F. Hutter, "Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[5] S. Han, J. Yoon, S. O. Arik, and T. Pfister, "Large language models can automatically engineer features for few-shot tabular learning," in *Proc. of the 41st Int'l Conf. on Machine Learning (ICML)*, 2024.

[6] N. Abhyankar, P. Shojaee, and C. K. Reddy, "LLM-FE: Automated feature engineering for tabular data with LLMs as evolutionary optimizers," *Transactions on Machine Learning Research (TMLR)*, 2025.

[7] Financial Action Task Force (FATF), "International standards on combating money laundering and the financing of terrorism & proliferation: The FATF recommendations," Updated 2023, 2012, available at https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Fatf-recommendations.html. Accessed: 2025-12-01.

[8] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," *arXiv preprint arXiv:1908.02591*, 2019.

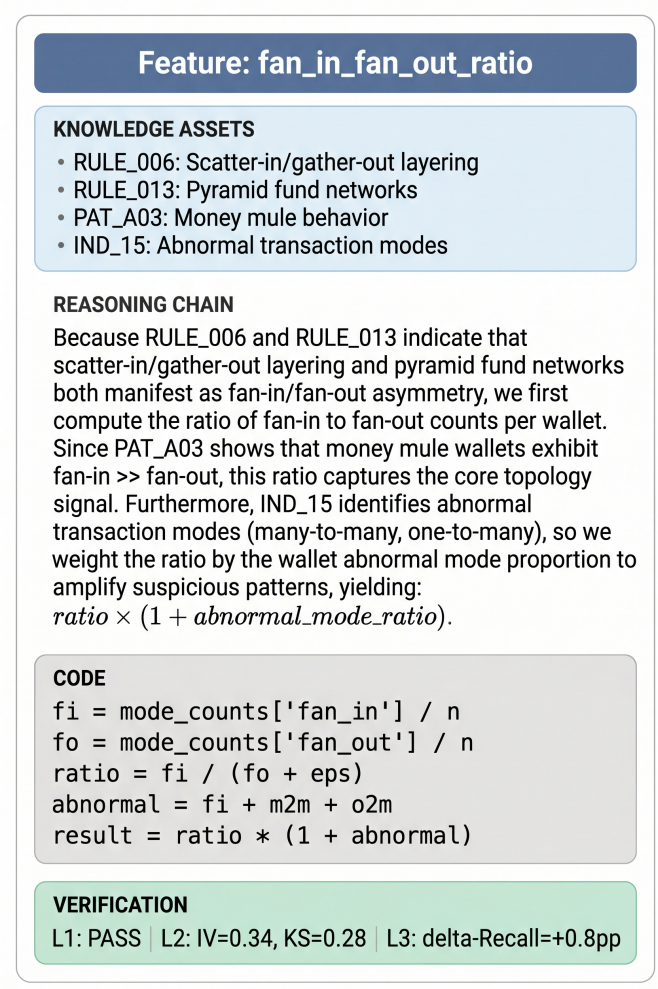


Fig. 2. Provenance card for `fan_in_fan_out_ratio`, recording knowledge assets, reasoning chain, generated code, and verification results. Each field enables regulatory audit.

[9] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *Proc. of the IEEE Int'l Conf. on Data Science and Advanced Analytics (DSAA)*, 2015, pp. 1–10.

[10] G. Katz, E. C. R. Shin, and D. Song, "ExploreKit: Automatic feature generation and selection," in *Proc. of the IEEE Int'l Conf. on Data Mining (ICDM)*, 2016, pp. 979–984.

[11] M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[12] C. Wang, Q. Wu, M. Weimer, and E. Zhu, "FLAML: A fast and lightweight AutoML library," *Proc. of Machine Learning and Systems (MLSys)*, vol. 3, pp. 434–447, 2021.

[13] F. Nargesian, H. Samulowitz, U. Khurana, E. B. Khalil, and D. S. Turaga, "Learning feature engineering for classification," in *Proc. of the 26th Int'l Joint Conf. on Artificial Intelligence (IJCAI)*, 2017, pp. 2529–2535.

[14] T. B. Brown, B. Mann, N. Ryder, M. Subbiah *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.

[15] H. Touvron, T. Lavril, G. Izacard, X. Martinet *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[16] H. Li, Y. Xu, F. Zhang, R. Shi, Z. Gu, Y. Yao, Z. Liu, and Z. Li, "RiskTagger: An LLM-based agent for automatic annotation of Web3 crypto laundering behaviors," *arXiv preprint arXiv:2504.17678*, 2025.

[17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. of the Int'l Conf. on Learning Representations (ICLR)*, 2023.

[18] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proc. of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[19] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" *Proc. of the Int'l Conf. on Learning Representations (ICLR)*, 2024.

[20] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu *et al.*, "Data interpreter: An LLM agent for data science," *arXiv preprint arXiv:2402.18679*, 2024.

[21] Z. Jiang, D. Schmidt, D. Srikanth, D. Xu, I. Kaplan, D. Jacenko, and Y. Wu, "AIDE: AI-driven exploration in the space of code," *arXiv preprint arXiv:2502.13138*, 2025.

[22] S. Guo, C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang, "DS-Agent: Automated data science by empowering large language models with case-based reasoning," in *Proc. of the 41st Int'l Conf. on Machine Learning (ICML)*, 2024.

[23] Z. Chen, L. D. Van Khoa, E. N. Teoh, A. Nazir, E. K. Karuppiah, and K. S. Lam, "Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: A review," *Knowledge and Information Systems*, vol. 57, no. 2, pp. 245–285, 2018.

[24] U. Kursuncu, M. Gaur, and A. Sheth, "Knowledge infused learning (K-IL): Towards deep incorporation of knowledge in deep learning," *Proc. of the AAAI Spring Symposium*, 2020.

[25] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *Journal of Big Data*, vol. 3, no. 1, pp. 1–40, 2016.

[26] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," *ACM SIGKDD Explorations Newsletter*, vol. 15, no. 2, pp. 49–60, 2014.

[27] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. of the 22nd ACM SIGKDD Int'l Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[28] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[29] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.